

«TÍTULO DE LA GUÍA»

«NOMBRE DE LA ASIGNATURA» — Unidad «N»

«Docente / Institución»

«Semestre / Fecha»

Objetivos de aprendizaje

Al finalizar esta guía, el estudiante será capaz de:

- Explicar qué es la recursividad y distinguir caso base de caso recursivo.
- Trazar la pila de llamadas de una función recursiva paso a paso.
- Implementar soluciones recursivas correctas en Python.
- Reconocer cuándo la recursión es preferible a la iteración y viceversa.

Conocimientos previos

Antes de comenzar, debes manejar: definición de funciones en Python, parámetros y valores de retorno, condicionales (`if/else`) y el concepto de variable local.

1 ¿Qué es la recursividad?

Una función **recursiva** es aquella que se llama a sí misma para resolver una versión más pequeña del mismo problema. La idea descansa en dos piezas que deben coexistir siempre.

Definición: recursividad

Técnica en la que la solución de un problema se expresa en términos de soluciones de instancias más pequeñas del mismo problema, hasta alcanzar un *caso base* que se resuelve directamente.

1.1 Las dos piezas obligatorias

1. **Caso base:** la condición de parada. Sin él, la función se llama infinitamente y agota la pila.
2. **Caso recursivo:** la llamada a sí misma con una entrada estrictamente más cercana al caso base.

Cuidado

Si el caso recursivo no *avanza* hacia el caso base (por ejemplo, llamar con el mismo argumento), obtendrás un `RecursionError` en Python.

1.2 Cómo razona la máquina: la pila de llamadas

Cada llamada pendiente se apila; cuando un caso base retorna, la pila se desapila resolviendo cada nivel. Comprender este apilamiento es la clave para depurar recursión.

2 Ejemplos resueltos

Ejemplo resuelto: factorial de un número

El factorial $n! = n \times (n - 1) \times \cdots \times 1$, con $0! = 1$.

```

1 def factorial(n):
2     # Caso base: el factorial de 0 es 1
3     if n == 0:
4         return 1
5     # Caso recursivo: n * factorial de (n-1)
6     return n * factorial(n - 1)
7
8 print(factorial(4)) # -> 24

```

Traza de factorial(4):

$$\begin{array}{c}
 4 \cdot 3 \cdot 2 \cdot 1 \cdot \underbrace{1}_{n=0} = 24 \\
 \underbrace{\hspace{1.5cm}}_{n=1} \\
 \underbrace{\hspace{1.5cm}}_{n=2} \\
 \underbrace{\hspace{1.5cm}}_{n=3}
 \end{array}$$

Nota

Observa que cada llamada reduce **n** en uno: así se garantiza que el caso base ($n = 0$) siempre se alcanza para entradas no negativas.

3 Ejercicios propuestos

Ejercicio 1 (básico)

Escribe una función recursiva `suma_hasta(n)` que devuelva $1 + 2 + \cdots + n$. Identifica explícitamente el caso base.

Ejercicio 2 (intermedio)

Implementa `potencia(base, exp)` de forma recursiva, sin usar el operador `**`. Asume `exp` entero no negativo.

Ejercicio 3 (desafío)

La sucesión de Fibonacci se define $F(0) = 0$, $F(1) = 1$, $F(n) = F(n - 1) + F(n - 2)$. Implementala recursivamente y luego responde: ¿por qué esta versión es ineficiente para n grande? Justifica en términos del número de llamadas.

Soluciones

Ejercicio 1.

```
1 def suma_hasta(n):  
2     if n == 0:          # caso base  
3         return 0  
4     return n + suma_hasta(n - 1)
```

Ejercicio 2.

```
1 def potencia(base, exp):  
2     if exp == 0:        # caso base: cualquier base^0 = 1  
3         return 1  
4     return base * potencia(base, exp - 1)
```

Ejercicio 3. La versión directa recalcula los mismos valores muchas veces: $F(n)$ genera dos llamadas, cada una otras dos, etc., dando un crecimiento exponencial $O(2^n)$ de llamadas. Se resuelve con memoización o con una versión iterativa $O(n)$.

Referencias

- «Autor, Título, Editorial, Año / URL con fecha de consulta»